
Workgroup: httpbis
Internet-Draft: draft-rajappa-httpbis-connection-contamination-02
Published: 13 August 2026
Intended Status: Best Current Practice
Expires: 14 February 2027
Author: M. Rajappa
IBM

Mitigating HTTP/3 Connection Contamination in Multi-Tenant and CDN-Fronted Deployments

Abstract

HTTP/3 [RFC9114] clients commonly reuse ("coalesce") an existing QUIC [RFC9000] connection for requests to a second origin when the TLS certificate presented on that connection is also valid for the second origin, even though the two origins may route to entirely different backends. This document describes "connection contamination," a class of security exposure that arises when a routing layer -- reverse proxy, load balancer, or CDN edge -- determines backend routing using a signal established at connection setup rather than re-validated per request. Under that condition, a coalesced connection can be used to reach an unintended backend origin, potentially enabling cross-tenant data leakage, authentication bypass, and response-queue interference analogous to HTTP request smuggling.

This document defines the underlying mechanism, characterizes the attacker model, distinguishes connection contamination from related QUIC exposures, and provides normative operational guidance for implementers and operators of HTTP/3-terminating infrastructure.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 14 February 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
2. Terminology	4
3. Applicability	5
4. Background: Connection Coalescing	5
4.1. Protocol Rules	5
4.2. Relationship to HTTP/2 Coalescing	6
4.3. Client Behavior Variance	6
5. Connection Contamination: Threat Description	6
5.1. Attacker Model	6
5.2. Threat Conditions	7
5.3. Attack Path	7
5.4. Potential Consequences	7
6. Recommendations	8
6.1. Per-Request Host Revalidation	8
6.2. Certificate Scope Minimization	8
6.3. SNI and :authority Consistency Enforcement	8
6.4. Trust-Boundary Segmentation	9
6.5. Cache Contamination Mitigation	9
6.6. Detection Guidance	9

7. Security Considerations	10
7.1. Client Behavior Must Not Be Assumed Stable	10
7.2. Connection Migration Interaction	10
7.3. 421 Misdirected Request as a Defense Mechanism	10
7.4. Scope of Normative Requirements	11
8. IANA Considerations	11
9. References	11
9.1. Normative References	11
9.2. Informative References	11
Acknowledgments	12
Changes from -00	12
Author's Address	12

1. Introduction

HTTP/3 [RFC9114] and its QUIC [RFC9000] transport depart from a long-standing assumption in HTTP/1.1- and HTTP/2-era infrastructure: that a network connection is uniquely and durably associated with a single origin for the lifetime of that connection, and that routing decisions made when the connection is established remain valid for every request subsequently carried on it.

To reduce handshake overhead, HTTP/3 clients are permitted to reuse an already-open connection for a request to a second origin when the TLS [RFC8446] certificate negotiated on that connection is also valid for the second origin -- for example via a Subject Alternative Name (SAN) or wildcard entry -- and the client considers the second origin's resolved address equivalent to the address already in use (see [RFC9114] Section 3.3). This behavior, known as connection coalescing, is a performance optimization with no inherent security implication at the protocol level.

Notably, [RFC9114] Section 3.3 already anticipates a server's need to reject connection reuse for a given origin, noting that a server "that does not wish clients to reuse HTTP/3 connections for a particular origin can indicate that it is not authoritative for a request by sending a 421 (Misdirected Request) status code" ([RFC9110] Section 7.4). This document's normative contribution is not a new protocol mechanism, but a recommendation that this existing, currently optional behavior become mandatory for the specific deployment pattern described in Section 4 -- where the operational cost of an occasional false rejection is substantially lower than the cost of silent cross-tenant data exposure.

A security-relevant side effect arises only when infrastructure in front of the origin -- a reverse proxy, load balancer, or CDN edge -- makes or caches a routing decision at connection establishment (or on the connection's first request) rather than re-evaluating the effective request host on every request the connection subsequently carries. Under that condition, a client holding a coalescing-eligible connection to one origin can direct a subsequent request on that same connection to a different origin sharing the same certificate, and have that request delivered to the second origin's backend even though the underlying transport and TLS session were established against the first. This document refers to that condition as "connection contamination," following the terminology introduced in [KETTLE2022].

Connection contamination is architecturally distinct from, though sharing a common root cause with, QUIC connection-migration abuse against stateful middleboxes as described in [COMSNETS2024]. The former is a request-routing and data-isolation exposure; the latter is a resource-exhaustion and availability exposure. This document addresses connection contamination only.

This document applies to multi-tenant SaaS deployments and CDN-fronted architectures in which two or more distinct origins are served behind a shared TLS certificate and a shared HTTP/3 listener. Single-origin deployments are outside its scope.

2. Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

The following terms are used throughout this document:

Origin: As defined in [RFC6454], the tuple of (scheme, host, port) that identifies a distinct web origin. Two origins are considered isolated when no intentional cross-origin resource sharing or routing is permitted between them.

Connection Coalescing: The HTTP/3 client behavior, specified in [RFC9114] Section 3.3, by which an existing QUIC connection may be reused for requests to a second origin when the certificate already negotiated on that connection is also valid for the second origin and the resolved addresses are considered equivalent.

Connection Contamination: The condition, introduced in [KETTLE2022] and defined precisely in Section 4 of this document, in which a coalesced request is delivered to an unintended backend origin due to stale or connection-scoped routing at a routing layer.

Routing Layer: A reverse proxy, load balancer, or CDN edge component that receives HTTP/3 connections from clients and forwards requests to one or more upstream backend origins based on request metadata.

Effective Request Host: The value of the `:authority` pseudo-header field as defined in [RFC9114] Section 4.3.1 (see also [RFC9110] Section 7.2 for the `Host/:authority` relationship) carried on an individual HTTP/3 request, which identifies the target origin for that request independently of any connection-level signal.

3. Applicability

This document applies to deployments meeting all of the following conditions:

1. One or more HTTP/3 listeners terminate connections on behalf of two or more distinct origins (as defined in [RFC6454]).
2. Those origins share a TLS certificate whose Subject Alternative Name (SAN) set or wildcard pattern covers more than one origin hostname.
3. A routing layer forwards requests from the shared listener to per-origin backends.

Deployments in which all origins sharing a certificate are intentionally permitted to share backend state (i.e., no isolation boundary exists) are outside the primary scope of Section 6, though the detection guidance in Section 6.5 may still be useful for confirming intentional behavior.

Single-origin deployments, deployments using per-origin certificates, and deployments where the HTTP/3 listener itself is the only backend (no upstream routing) are out of scope.

4. Background: Connection Coalescing

4.1. Protocol Rules

HTTP/3 clients determine coalescing eligibility per [RFC9114] Section 3.3 using two criteria applied at the transport/TLS layer:

1. The TLS certificate already negotiated on an open connection is valid for the second origin's hostname (via SAN or wildcard match).
2. DNS resolution for the second origin's hostname yields an IP address that the client's stack considers equivalent to the address already in use for the existing connection.

The standard does not mandate strict IP-address equality as the sole definition of "equivalent address"; implementations MAY apply their own equivalence criteria. Infrastructure MUST NOT assume that a given HTTP/3 connection will carry requests for only the origin used to establish it.

4.2. Relationship to HTTP/2 Coalescing

HTTP/2 [RFC9113] Section 9.1.1 defines a structurally identical coalescing mechanism: a client MAY reuse an existing connection for a second origin when the server's TLS certificate is valid for that origin and the same IP address is in use. The same stale-routing exposure described in this document therefore applies to HTTP/2 deployments sharing certificates across isolation boundaries. The present document is scoped to HTTP/3 for two reasons:

1. QUIC's UDP-based transport and built-in connection migration ([RFC9000] Section 9) make it significantly harder for intermediaries to correlate a given packet flow with the origin that established the original connection, increasing the opacity of coalescing to infrastructure.
2. HTTP/3 deployments are more likely to aggregate multiple tenants behind a single QUIC listener at a CDN or cloud edge, amplifying the blast radius when a routing layer misconfiguration is present.

Operators of HTTP/2 infrastructure sharing certificates across isolation boundaries SHOULD apply the per-request host revalidation guidance in Section 6.1 equally to HTTP/2 connections. A parallel BCP addressing the HTTP/2 case may be warranted; that work is outside the scope of this document.

4.3. Client Behavior Variance

Client behavior with respect to coalescing-eligibility strictness is implementation-defined. Different browser and library implementations may apply different levels of address-equivalence checking, and this behavior may change across software versions. Server-side mitigations defined in Section 6 MUST NOT depend on any particular client-side coalescing behavior remaining constant (see also Section 7).

5. Connection Contamination: Threat Description

5.1. Attacker Model

The attacker is assumed to be a party able to open an HTTP/3 connection to at least one origin in a shared-certificate set, but without authorization to access one or more other origins covered by the same certificate. This encompasses:

- A co-tenant in a multi-tenant SaaS deployment who holds legitimate credentials for their own tenant origin but not for other tenants.
- An external client who can resolve and reach a lower-trust origin (e.g., a marketing or staging hostname) that shares a wildcard certificate with a higher-trust origin (e.g., an authentication or payment endpoint).
- An attacker performing targeted cross-origin exfiltration by constructing coalescing-eligible connections before issuing a forged-authority request.

The attacker does not need to compromise TLS, QUIC, or any cryptographic primitive. The attack is entirely at the HTTP routing layer and requires only a valid HTTP/3 client.

5.2. Threat Conditions

Connection contamination requires all three of the following conditions to hold simultaneously:

1. A TLS certificate whose scope, via wildcard or multi-SAN issuance, covers two or more origins that are intended to be logically or organizationally isolated from one another.
2. A client willing to coalesce an existing HTTP/3 connection onto a second origin covered by that certificate, per Section 4.
3. A routing layer that determines backend routing using a signal other than the effective request host (`:authority`) re-evaluated on every individual request -- for example, a cached association formed from the connection's first request, or a routing key read once from the TLS SNI or QUIC connection metadata rather than per request.

5.3. Attack Path

The following illustrates a typical contamination scenario in a multi-tenant CDN-fronted deployment:

```
Step 1 - Legitimate connection establishment:
  Client —[QUIC handshake, SNI=tenant-a.example.com]—> CDN Edge
  CDN Edge: stores routing key "backend-A" keyed on connection ID
  CDN Edge —> backend-A

Step 2 - Coalescing-eligible second request:
  Client (same connection): :authority = tenant-b.example.com
  CDN Edge: looks up routing key by connection ID → "backend-A"
  CDN Edge —> backend-A    ← WRONG: should route to backend-B

Result:
  Tenant B's request is served by backend-A.
  Tenant A's response queue may be interleaved with Tenant B's traffic.
```

5.4. Potential Consequences

Where the three conditions in Section 5.2 hold, the following effects are possible:

- **Cross-tenant data leakage:** Responses intended for one backend may be returned to a client of a different backend if response queues are shared or not strictly isolated per request.
- **Authentication bypass:** If the contaminated backend performs authorization checks relative to the connection's original SNI rather than the request's `:authority`, a lower-privileged client may access higher-privileged resources.
- **Cache contamination:** A CDN or shared cache layer may store a response under a cache key derived from the victim origin, poisoning subsequent legitimate requests to that origin. Mitigations for this impact are addressed in Section 6.5.

- **Response queue interference:** Analogous to HTTP request smuggling [REQUESTSMUGGLING], interleaved responses on a shared backend connection can cause response desynchronization.

The underlying mechanism here is QUIC-specific connection reuse and stale routing, distinct from the ambiguous header parsing that enables classical HTTP request smuggling [REQUESTSMUGGLING].

6. Recommendations

The following recommendations apply to operators and implementers of HTTP/3-terminating infrastructure -- reverse proxies, load balancers, and CDN edges -- that front more than one origin behind a shared TLS certificate.

6.1. Per-Request Host Revalidation

A routing layer terminating HTTP/3 connections MUST re-evaluate the effective request host -- the value of the `:authority` pseudo-header field ([RFC9110] Section 7.2) -- on every individual request received on a connection, and MUST NOT rely solely on a routing decision cached from connection establishment, from the TLS SNI negotiated at handshake, or from an earlier request on the same connection.

Where a routing layer determines that a request's `:authority` value does not correspond to an origin it can properly serve on the connection on which the request arrived, it MUST respond with 421 (Misdirected Request) ([RFC9110] Sections 7.4 and 15.5.20) rather than forwarding the request to an incorrect backend.

These requirements reflect current practice among major HTTP/3 reverse proxy implementations (e.g., Nginx, Envoy, and Caddy each perform per-request `:authority` evaluation by default when virtual-host routing is configured) and are consistent with the server behavior already anticipated by [RFC9114] Section 3.3. The MUST level is warranted because silent incorrect routing -- the failure mode in the absence of this requirement -- constitutes a data-isolation breach with no observable signal to the affected tenant.

6.2. Certificate Scope Minimization

Operators SHOULD NOT deploy a TLS certificate whose SAN or wildcard scope spans origins with differing trust levels or isolation requirements. Where broad-scope certificates are operationally necessary (for example, in multi-tenant SaaS hosting or large-scale CDN deployments), the routing layer behind that certificate MUST implement per-request host revalidation as described in Section 6.1 without exception.

6.3. SNI and `:authority` Consistency Enforcement

A routing layer SHOULD compare the TLS SNI value presented at handshake with the `:authority` pseudo-header of each subsequent request on the same connection. A mismatch between SNI and `:authority` is a reliable indicator of a coalesced request and SHOULD trigger

revalidation logic. Where the routing layer cannot serve the `:authority` value on the current connection, it MUST respond with 421 (Misdirected Request) per [RFC9110] Section 15.5.20, allowing compliant clients to retry on a new connection.

6.4. Trust-Boundary Segmentation

Where isolation requirements differ significantly between origins (for example, an origin handling authentication or payment functions versus a marketing or staging origin), operators SHOULD segment HTTP/3 listeners such that high-isolation origins are not reachable via a certificate shared with lower-trust origins. This segmentation SHOULD be implemented as defense in depth, independent of whether Section 6.1 is correctly implemented.

6.5. Cache Contamination Mitigation

When a routing layer is co-located with or upstream of a shared cache, the following additional controls MUST or SHOULD be applied to prevent cache contamination (one of the consequences identified in Section 5.4):

1. Cache keys MUST include the effective request host (`:authority` value) and MUST NOT be derived solely from connection-level metadata such as the TLS SNI or QUIC connection ID.
2. Routing layers SHOULD propagate a verified, normalized host value to upstream caches as part of the forwarded request rather than relying on the cache to re-derive it independently.
3. Cache entries associated with a given origin MUST NOT be served in response to a request whose `:authority` names a different origin, even when both origins are covered by the same TLS certificate.

Operators SHOULD audit cache-key configuration on shared caches whenever per-request host revalidation (Section 6.1) is first deployed, as correcting routing behavior without correcting cache-key derivation leaves the cache contamination consequence unmitigated.

6.6. Detection Guidance

Operators assessing existing infrastructure for this exposure SHOULD:

1. Enumerate the full SAN and wildcard scope of each TLS certificate served on HTTP/3 listeners in scope.
2. For each pair of hostnames sharing a certificate, determine whether a client would consider them coalescing-eligible per Section 4 and [RFC9114] Section 3.3.
3. For each coalescing-eligible pair, verify -- via authorized testing only -- that a request specifying the second hostname's `:authority`, issued on a connection established against the first hostname, is either:
 - Rejected with 421 (Misdirected Request) per [RFC9110] Section 15.5.20, or
 - Independently re-routed to the correct backend for the second hostname.

4. Confirm that no routing state keyed on connection ID, QUIC connection metadata, or TLS session parameters persists across requests in a way that could override per-request :authority evaluation.
5. Verify that cache keys on any shared cache layer include the effective request host and cannot be collided across tenant origins.

Operators performing this verification SHOULD use non-destructive, read-only requests and MUST NOT perform this testing against infrastructure without explicit authorization from the system owner.

7. Security Considerations

This entire document concerns a security exposure and its mitigation; Section 6 constitutes its primary normative guidance.

7.1. Client Behavior Must Not Be Assumed Stable

Browser and library client behavior with respect to the strictness of coalescing-eligibility evaluation (Section 4.1) is an implementation choice that is not guaranteed by [\[RFC9114\]](#) or [\[RFC9000\]](#) and may vary across client versions. Server-side mitigations per Section 6 MUST NOT depend on any particular client-side coalescing behavior remaining constant. A more permissive future client implementation would silently expand the exposure window for infrastructure that relied on current client behavior as a mitigating control.

7.2. Connection Migration Interaction

QUIC connection migration ([\[RFC9000\]](#) Section 9) allows a QUIC connection to survive changes in the client's network path (e.g., switching from Wi-Fi to cellular). Migration does not materially change the threat surface described in this document: the connection and its associated TLS session remain the same object, coalescing eligibility criteria are unaffected, and routing-layer state keyed on connection ID survives the migration. Operators SHOULD NOT treat connection migration as a mitigating factor for connection contamination.

7.3. 421 Misdirected Request as a Defense Mechanism

The 421 (Misdirected Request) status code defined in [\[RFC9110\]](#) Section 15.5.20 was introduced specifically to handle the case in which a server receives a request on a connection it cannot properly serve for that target origin. [\[RFC9114\]](#) Section 3.3 already identifies this status code as the mechanism by which a server opts out of connection reuse for a given origin – this document does not introduce new protocol behavior, but elevates an existing, currently discretionary ("can indicate") mechanism to a normative requirement (Section 6.1) for the specific class of deployment described in Section 4, where the isolation stakes of an incorrect reuse decision are high. Correct implementation of 421 responses at routing layers is therefore not merely a best practice but the base specification's own designated defense, and is the primary normative defense against connection contamination described in this document. Implementations that suppress or translate 421 responses negate this defense.

7.4. Scope of Normative Requirements

The normative requirements in Section 6 are addressed to routing-layer implementers and operators. They do not impose requirements on HTTP/3 client implementations, which are governed by [RFC9114] and [RFC9000].

8. IANA Considerations

This document has no IANA actions.

9. References

9.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/rfc/rfc2119>>.
- [RFC6454] Barth, A., "The Web Origin Concept", RFC 6454, DOI 10.17487/RFC6454, December 2011, <<https://www.rfc-editor.org/rfc/rfc6454>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/rfc/rfc8174>>.
- [RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, DOI 10.17487/RFC8446, August 2018, <<https://www.rfc-editor.org/rfc/rfc8446>>.
- [RFC9000] Iyengar, J., Ed. and M. Thomson, Ed., "QUIC: A UDP-Based Multiplexed and Secure Transport", RFC 9000, DOI 10.17487/RFC9000, May 2021, <<https://www.rfc-editor.org/rfc/rfc9000>>.
- [RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <<https://www.rfc-editor.org/rfc/rfc9110>>.
- [RFC9114] Bishop, M., Ed., "HTTP/3", RFC 9114, DOI 10.17487/RFC9114, June 2022, <<https://www.rfc-editor.org/rfc/rfc9114>>.

9.2. Informative References

- [COMSNETS2024] Hari Hara Sudhan S, H.H. and S. G. Kulkarni, "Security and Service Vulnerabilities with HTTP/3", 2024 16th International Conference on COMMunication Systems & NETworkS (COMSNETS), Bengaluru, India pp. 55-60, doi: 10.1109/COMSNETS59351.2024.10427406, 2024, <<https://ieeexplore.ieee.org/document/10427406>>.

- [KETTLE2022]** Kettle, J., "HTTP/3 connection contamination: an upcoming threat?", 2022, <<https://portswigger.net/research/http-3-connection-contamination>>. Original public research introducing the HTTP/3 connection contamination vulnerability class discussed throughout this document.
- [REQUESTSMUGGLING]** Kettle, J., "HTTP Request Smuggling", DEF CON 27 AppSec Village Las Vegas, NV, USA, 2019, <<https://www.defcon.org/html/defcon-27/dc-27-speakers.html#Kettle>>. Original public presentation of HTTP request-smuggling techniques; referenced here to establish analogy with response-queue interference described in Section 5.4.
- [RFC9113]** Thomson, M., Ed. and C. Benfield, Ed., "HTTP/2", RFC 9113, DOI 10.17487/RFC9113, June 2022, <<https://www.rfc-editor.org/rfc/rfc9113>>.

Acknowledgments

The author acknowledges the original public research on this exposure class by James Kettle of PortSwigger Research [KETTLE2022], on which the threat description in Section 5 draws directly. The author also acknowledges [COMSNETS2024] for characterizing related QUIC transport-layer security vulnerabilities in multi-tenant contexts.

Changes from -00

-01 added the Applicability section (Section 3), the Attacker Model (Section 5.1) with concrete actor scenarios, an illustrated Attack Path walkthrough (Section 5.3), the SNI and :authority Consistency Enforcement recommendation (Section 6.3), the Cache Contamination Mitigation recommendation (Section 6.5), the Connection Migration Interaction discussion (Section 7.2), and explicit grounding of the Section 6.1 MUST requirement in the pre-existing, currently discretionary mechanism already described in [RFC9114] Section 3.3 (Section 7.3). Reference-section citations for [RFC9114], [RFC9110], and [RFC9113] were corrected to their proper section numbers throughout.

-02 corrected two reference errors: the [KETTLE2022] entry was updated to cite the original PortSwigger research post that introduced the connection contamination terminology used throughout this document, replacing an incorrectly titled prior entry; the [COMSNETS2024] author list was corrected to list only the paper's actual two authors.

Author's Address

Madhusudhan Rajappa

IBM

Email: madhu.sudhan@in.ibm.com